

Resumen

El proyecto consiste en el diseño y desarrollo de un **Marketplace Integral de Servicios y Productos**, estructurado bajo una **arquitectura pura de microservicios**. La plataforma actúa como un ecosistema centralizado que conecta de manera eficiente la oferta y la demanda en tiempo real, abarcando múltiples verticales de negocio como logística (delivery), transporte, mantenimiento del hogar (limpieza, gasfitería) y comercio electrónico (compra y venta de artículos).

Descripción Técnica y Arquitectónica

Para garantizar una **escalabilidad masiva, alta disponibilidad y tolerancia a fallos** — requisitos indispensables para una plataforma de alcance global—, el sistema abandona el enfoque monolítico tradicional. En su lugar, se implementa una arquitectura distribuida donde cada dominio del negocio opera como un servicio independiente.

Esto permite:

- **Despliegue y escalamiento independiente:** Si el módulo de "Delivery" experimenta un pico de tráfico, se pueden asignar más recursos exclusivamente a ese servicio sin afectar al módulo de "Venta de productos".
 - **Agnosticismo tecnológico:** La separación de dominios (Usuarios, Pagos, Geolocalización, Mensajería) permite que cada equipo de desarrollo utilice las tecnologías y bases de datos más eficientes para cada problema específico.
- **Resiliencia del sistema:** La caída de un servicio no crítico (como el motor de recomendaciones) no interrumpe el flujo principal de transacciones de la plataforma.

Propuesta de Valor y Visión a Gran Escala

A diferencia de los clasificados tradicionales, esta plataforma no es solo un tablón de anuncios, sino un **gestor transaccional completo (End-to-End)**.

La visión del producto integra:

1. **Seguridad y Confianza:** A través de sistemas robustos de gestión de identidad, perfiles verificados y un motor de calificaciones asimétrico.
2. **Transaccionalidad Integrada:** Gestión de pagos, retenciones (escrow) y comisiones automatizadas dentro del mismo ecosistema.
3. **Comunicación en Tiempo Real:** Interacción fluida entre clientes y proveedores mediante mensajería instantánea y notificaciones push, garantizando una experiencia de usuario (UX) sin fricciones.
4. **Descubrimiento Inteligente:** Motores de búsqueda avanzados y geolocalización para conectar a los usuarios con los servicios más relevantes y cercanos a su ubicación.

1. El Núcleo de la Plataforma (Lo que ya tienen, pero optimizado)

- **Microservicio de Identidad y Usuarios (IAM):** Va más allá de iniciar sesión. Debe manejar la validación de identidad (pedir fotos del carnet para los conductores o limpiadores) para generar confianza, roles (comprador, vendedor, repartidor) y la seguridad de las cuentas.
- **Microservicio de Catálogo y Publicaciones:** Les sugiero fusionar "Categorías" aquí. Este servicio se encargará de crear, editar y categorizar las ofertas ("ofrezco pasear perros") y las demandas ("necesito un gasfiter").
- **Microservicio de Búsqueda y Descubrimiento:** Este es vital. En el mundo real, este microservicio usaría motores avanzados (como Elasticsearch) para que las búsquedas sean ultrarrápidas, toleren faltas de ortografía y filtren por rango de precio y ubicación exacta.
- **Microservicio de Reputación (Calificaciones):** El corazón de la confianza. Debe gestionar las reseñas, los promedios de estrellas y, en una visión grande, detectar reseñas falsas.
- **Microservicio de Soporte y Disputas (Servicio al Cliente):** Gestión de tickets, devoluciones de dinero y mediación entre usuarios cuando algo sale mal.

2. Los Motores de Interacción (Comunicación en tiempo real)

- **Microservicio de Mensajería:** Un chat en tiempo real (usando WebSockets). Debe ser independiente porque el volumen de datos de un chat constante saturaría otros servicios.
- **Microservicio de Notificaciones:** Un motor asíncrono que envía correos, SMS o notificaciones push al celular ("Tu repartidor está llegando", "Tienes un nuevo mensaje").

3. Los Microservicios "Trillonarios" (Lo que les falta para ser gigantes)

Para que su proyecto realmente destaque como una idea multimillonaria, necesitan agregar estos componentes clave que separan a un simple foro de una plataforma como Uber o MercadoLibre:

- **Microservicio de Pagos y Billetera (Payments):** ¡Aquí está el negocio! Necesitan un servicio que procese tarjetas de crédito, cobre una comisión por cada transacción, y tenga un sistema de "Escrow" (retener el dinero de forma segura hasta que el cliente confirme que recibió el servicio de limpieza o su comida).
- **Microservicio de Geolocalización y Rutas:** Si van a tener categorías tipo "Delivery" o "Uber", necesitan un servicio dedicado exclusivamente a rastrear ubicaciones en tiempo real, calcular distancias y emparejar al conductor más cercano con el cliente.
- **Microservicio de Matchmaking (Recomendaciones):** Un servicio que analice los datos para sugerir cosas. Si alguien contrata servicios de limpieza, el sistema automáticamente le recomienda servicios de mantenimiento de hogar.

TECNOLOGÍAS.

1. Frontend

- **Tecnología principal: React** (usando Vite) o **Next.js**
 - *Por qué:* Es el estándar de la industria, la documentación es gigantesca y hay miles de componentes ya hechos. Si desean que sea una app móvil, pueden usar **React Native**.
- **Librerías UI (Para avanzar rápido con el diseño): Tailwind CSS + shadcn/ui o Chakra UI.** Les permitirá construir pantallas profesionales sin perder días diseñando botones o formularios desde cero.

2. Backend (Los Microservicios)

Les recomiendo dividir los backend entre dos lenguajes muy populares y ágiles:

- **Node.js (NestJS o Express) – Para la mayoría de los servicios:**
 - *Uso recomendado:* Microservicios de **Mensajería, Notificaciones, Gateway y Usuarios**.
 - *Por qué:* NestJS da una estructura muy ordenada (orientada a arquitectura de software), maneja WebSockets de forma nativa para el chat en vivo y utiliza TypeScript.

3. Bases de Datos (Persistencia por Microservicio)

En una arquitectura de microservicios real, **cada servicio maneja su propia base de datos**.

- **PostgreSQL + PrismaORM**

4. Infraestructura, Orquestación y Comunicación

- **API Gateway (La Puerta de Entrada): Nginx** o un Gateway propio en NestJS/Express.
 - *Función:* El Frontend solo se conecta a un punto central (el Gateway), y este redirige las solicitudes al microservicio correspondiente.
- **Contenedores: Docker y Docker Compose** (*¡Infaltable!*).
 - *Función:* Permite que cualquier integrante del equipo levante toda la arquitectura de 10+ microservicios y bases de datos en su propia computadora con un solo comando: **docker compose up**.
- **Comunicación asíncrona: RabbitMQ**
 - *Función:* Para que los microservicios se hablen por eventos sin congelar la app. Ejemplo: Cuando se aprueba un pago, el microservicio de Pagos publica un mensaje en RabbitMQ y el microservicio de Notificaciones lo lee y envía el correo.

5. Servicios de Terceros e Integraciones

- **Pagos:** MercadoPago API (Modo Sandbox/Pruebas) o Stripe.
- **Archivos / Imágenes:** Cloudinary (módulo para subir imágenes de usuarios y productos directamente a la nube).
- **Mapas y Ubicación:** Mapbox API o Google Maps JavaScript API.
- **Hosting gratuito o de bajo costo para demo:** Render / Railway (Backend), Vercel (Frontend) y Supabase / MongoDB Atlas (Bases de datos).

PROPUESTAS PARA MEJORAR PROYECTO

1. Sistema de calendario y agendamiento
2. Portafolio Multimedia y Videos
3. formulario inicial de preferencias de servicios o trabajo
4. Verificación de identidad y reputación, perfiles verificados
5. Anuncios patrocinados / publicidad local

1. Sistema de calendario y agendamiento

- **Veredicto:** ☐ Microservicio Nuevo (**MS Booking**)
- **Cómo se integra:** Este dominio es lo suficientemente complejo para tener su propia base de datos (**booking_db**). Se encargará de manejar bloques horarios, disponibilidad del proveedor y zonas horarias.
- **Interacción:** Cuando el cliente intente crear una orden, **MS Órdenes** le preguntará a **MS Booking** si la fecha está libre. Si está libre, se bloquea el calendario; si se cancela el pago, se libera la hora.

2. Portafolio Multimedia y Videos

- **Veredicto:** ☐ Integración en Microservicio Existente (**MS Catálogo**)
- **Cómo se integra:** No necesitas un microservicio nuevo para esto. El **schema.prisma** del Catálogo simplemente se expande para incluir una tabla de **Galeria_Portafolio**.
- **Interacción:** Los videos e imágenes pesadas no se guardan en tu PostgreSQL, se suben a tu servicio externo **Cloudinary**. Tu **MS Catálogo** solo guarda las URLs de esos videos.

3. Formulario inicial de preferencias (Onboarding)

- **Veredicto:** ☐ Integración en Microservicio Existente (**MS Match y IAM**)
- **Interacción:** A nivel visual es una pantalla interactiva en **Next.js**. Cuando el usuario guarda el formulario, el Frontend envía un evento a **MS IAM** para marcar que el usuario completó el onboarding, y los datos de preferencias se guardan directamente

en la base de datos de **MS Match** para entrenar el algoritmo desde el día uno.

4. Verificación de identidad y reputación (Perfiles Verificados)

- **Veredicto:** ☐ Integración en Microservicio Existente (**MS IAM**)
- **Cómo se integra:** La verificación de identidad (KYC) es pura responsabilidad de gestión de usuarios.
- **Interacción:** **MS IAM** recibirá las fotos del carnet de identidad (que se suben a Cloudinary) y cambiará el estado del usuario a **kyc_verified: true**. Luego, el Frontend mostrará el "Ticket Azul" (Blue Check) en el perfil basándose en este estado. La parte de reputación ya la tienes cubierta con el **MS Reputation**.

5. Anuncios patrocinados / publicidad local

- **Veredicto:** ☐ Microservicio Nuevo (**MS Ads** o **MS Promoted**)
- **Cómo se integra:** Esta es tu carta ganadora para demostrarle al profesor que el proyecto es "grande". Monetizar a través de anuncios requiere una lógica totalmente separada de las órdenes normales.
- **Interacción:** Este microservicio manejará las campañas publicitarias (ej. "Quiero pagar \$10 para salir primero esta semana"). Se comunicará con **MS Pagos** para cobrar la campaña, y luego le avisará a **MS Search** (Elasticsearch) para que infle artificialmente la posición de ese proveedor en los resultados de búsqueda.